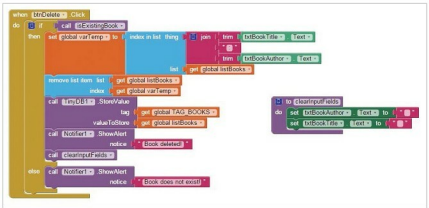

Figure 10: Block Editor Image 6

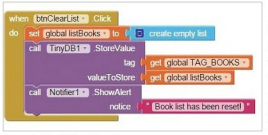

Figure 11: Block Editor Image 7

installation, you will see the icon of your application in the menu of your mobile. Here, you will see the default icon that can be changed, and I will tell you how to do this as we move ahead in this course.

I hope your application is working based on the requirements you have given. Now, depending upon your usability and customisation, you can change various things like images, sound and behaviour also.

Debugging the application

We have just created the prototype of the application with very basic functionality. What else might the user of your app be interested in? Give the various use cases that your app should be able to operate in some serious thought, so as not to annoy the user. Consider the following cases:

- Wouldn't it be nice to add some data validation upon entering the data in lower or upper case?
- Should we consider adding the cover image of the book as well?

These are some of the updates you could consider, and users may be pretty happy seeing these implemented. Think about other possible scenarios, and how you can integrate these into the application. Do ask me if you fail to accomplish any of the above cases.

You have successfully built another useful Android app for yourself. Happy inventing! END

By: Meghraj Singh Beniwal

The author, who is a freelance writer and an Android app developer, has a B. Tech in electronics and communication. He is currently working as an automation engineer at Infosys, Pune. He can be contacted at *meghrajsingh01@rediffmail.com* or *meghrajwithandroid@gmail.com*.

Continued from Page 61...

Figures 2 to 5 show a few API end point tests while Figures 2 and 3 show *POST/register* and *GET/task* APIs, Figures 4 and 5 show *POST/task* and *PUT/task/1* APIs.

In this article, you implemented a simple TaskAPI with Django and the Django REST framework, with basic

```
POST /register

HTTP 201 Created
Allow: POST, OPTIONS
Content-Type: application/json
Vary: Accept

{
    "username": "Demo"
}
```

Figure 2: Register user

```
GET /task/

HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

[]
```

Figure 3: List task

Figure 4: Create task

Figure 5: Update task

authentications and permissions. We have now learned quite a lot about the Django REST framework, including how to implement a Web-browsable API which can return JSON for you, how to configure serialisers to compose and transform your data, and how to use class based views to extract boilerplate code. END

References

[1] *http://www.django-rest-framework.org/#tutorial*
[2] *https://docs.djangoproject.com/en/1.9/*
[3] Source code related to this article is available at: *http://opensourceforu.com/article_source_code/aug16/djangoandrestapi.zip*

By: Yogendra Sharma

The author is a Java developer with a background in Python. He currently works in Pune at Siemens Industry Software Pvt Ltd as a product development engineer. His LinkedIn profile is available at *http://in.linkedin.com/in/yogendra0sharma*.